

FormulaCarDigitalTwin Stato del progetto

Umberto

Agosto 2026

- 1. Obiettivo generale
- 2. GitHub e struttura di sviluppo
- 3. Problema iniziale del percorso
- 4. Identità Git
- 5. Struttura delle directory
- 6. .gitignore
- 7. Ambiente Python
- 8. Virtual environment
- 9. pyproject.toml
- 10. Primo test
- 11. Package Python
- 12. FDT01 — Baseline Vehicle
- 13. Parametri della baseline
- 14. Parametri TBD
- 15. ParameterRecord
- 16. Database SQLite
- 17. Perché configuration_version
- 18. Repository layer
- 19. Importer
- 20. Script di inizializzazione
- 21. Problema incontrato nel database
- 22. Pint — sistema delle unità
- 23. Validation framework
- 24. Stato attuale dei test
- 25. Cosa abbiamo costruito realmente
- 26. Cosa NON abbiamo ancora fatto
- 27. Il prossimo livello identificato: Engineering Requirements
- 28. Architettura futura
- 29. Il concetto fondamentale del progetto
- 30. Stato attuale in una frase
- 31. Dove siamo nella roadmap
- 32. In sintesi

1. Obiettivo generale

L'obiettivo del progetto è costruire un **Digital Twin** di una vettura Formula, partendo da una configurazione iniziale tipo Formula 4 e sviluppandola progressivamente tramite:

- modellazione del veicolo
- database ingegneristico
- simulazioni
- validazione
- analisi delle prestazioni
- ottimizzazione
- confronto tra diverse configurazioni della vettura

L'idea fondamentale è che non vogliamo creare semplicemente una serie di script MATLAB/Python separati. Vogliamo costruire una vera **infrastruttura software di engineering**:

```
Requirements
↓
Vehicle configuration
```

```
↓
Engineering database
↓
Physical validation
↓
Models
↓
Simulation
↓
Results
↓
Validation
↓
Vehicle evolution
```

La prima vettura è **FDT01** e la prima configurazione è **FDT01 V1.0 — Baseline**.

2. GitHub e struttura di sviluppo

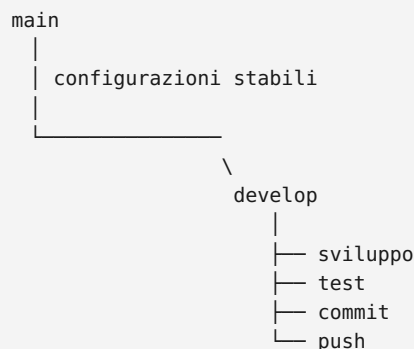
Repository GitHub: **Formula_digitaltwin**.

Struttura di sviluppo basata su Git, con branch `develop` collegato al branch remoto `origin/develop`.

Procedura eseguita:

```
cd C:\Users\umber\Formula_digitaltwin
git status
git checkout -b develop
git push -u origin develop
```

Concetto adottato:



3. Problema iniziale del percorso

All'inizio il percorso cercato era `cd FormulaCarDigitalTwin`, ma Windows rispondeva `Impossibile trovare il percorso specificato`.

Verificando la directory `C:\Users\umber` è emerso che il repository reale si trovava in:

```
C:\Users\umber\Formula_digitaltwin
```

4. Identità Git

Al primo commit Git aveva restituito `Author identity unknown`, perché non erano configurati `user.name` e `user.email`. Il problema è stato risolto, consentendo poi commit corretti.

5. Struttura delle directory

Struttura generale del progetto:

```
Formula_digitaltwin/
|
├── assets/
├── cad/
├── cfd/
├── data/
├── docs/
├── fem/
├── manufacturing/
├── notebooks/
├── scripts/
├── src/
├── tests/
├── validation/
|
├── .gitignore
├── README.md
├── pyproject.toml
└── .venv/
```

Funzione delle cartelle:

Cartella	Funzione
src/	Vero codice Python del framework
tests/	Test automatici
data/	Dati persistenti del progetto, incluso il database
docs/	Documentazione tecnica
scripts/	Script eseguibili per operazioni specifiche
cad/	Materiale relativo al CAD
cfd/	Materiale relativo alla CFD
fem/	Materiale relativo alla FEM
manufacturing/	Materiale relativo alla produzione
notebooks/	Jupyter notebook per analisi esplorative
validation/	Materiale destinato alla validazione dei modelli
assets/	Materiale di supporto

6. .gitignore

`.gitignore` configurato per evitare di versionare elementi che non devono finire su GitHub. In particolare, `.venv/` non deve essere caricato nel repository (il virtual environment è locale alla macchina).

7. Ambiente Python

Python disponibile sul computer: **3.12.7**.

Inizialmente `python --version` non funzionava correttamente a causa dell'alias di Windows. Usando `py --version` si è ottenuto correttamente `Python 3.12.7`.

8. Virtual environment

Creato `.venv/` nella root del progetto. Risolto anche il problema iniziale di PowerShell relativo alla policy di esecuzione.

Attivazione:

```
.\.venv\Scripts\Activate.ps1
```

Prompt risultante:

```
(.venv) PS C:\Users\umber\Formula_digitaltwin>
```

Verifiche:

```
python --version      → Python 3.12.7
python -m pip --version → pip installato dentro .venv
```

9. pyproject.toml

Il progetto Python è impostato in modo strutturato tramite `pyproject.toml`, che fornisce:

- configurazione del progetto
- dipendenze
- configurazione pytest
- struttura Python moderna

Il test runner riconosce:

```
rootdir: C:\Users\umber\Formula_digitaltwin
configfile: pyproject.toml
testpaths: tests
```

10. Primo test

Primo test per verificare che il package fosse correttamente importabile: risultato iniziale **1 passed**. Il framework è poi cresciuto progressivamente fino a **16 passed** — test suite funzionante e completamente verde.

11. Package Python

Package principale: `src/fdt/` (fdt = *Formula Digital Twin*).

Struttura concettuale:

```
fdt/
├──
├── database/
├── units/
├── validation/
├── vehicle/
└── ...
```

12. FDT01 — Baseline Vehicle

Primo veicolo definito: **FDT01**, configurazione **FDT01 V1.0**, nome **Baseline**.

Obiettivo: avere una vettura Formula 4 di riferimento — non la macchina perfetta, ma una baseline ingegneristicamente plausibile su cui far evolvere il progetto.

13. Parametri della baseline

Mass

Parametro	Valore
mass.mass	530 kg
mass.cg_height	0.30 m

mass.cg_longitudinal_position	1.485 m
mass.cg_lateral_position	0.0 m
mass.front_mass_distribution	0.45

Geometry

Parametro	Valore
geometry.wheelbase	2.70 m
geometry.front_track	1.50 m
geometry.rear_track	1.48 m
geometry.ground_clearance	0.05 m
geometry.tire_radius	0.28 m

Powertrain

Parametro	Valore
powertrain.maximum_power	130000 W
powertrain.maximum_torque	220 N·m
powertrain.maximum_engine_speed	10000 rpm
powertrain.number_of_gears	6
powertrain.drivetrain_efficiency	0.90

Tires

Parametro	Valore
tires.diameter	0.66 m
tires.front_width	0.22 m
tires.rear_width	0.26 m

Brakes

Parametro	Valore
brakes.brake_bias	0.60

Suspension

Parametro	Valore
suspension.front_type	Double wishbone / pushrod
suspension.rear_type	Double wishbone / pushrod

14. Parametri TBD

Decisione importante: **non inventare dati che non conosciamo ancora**. Parametri come `drag_coefficient`, `lift_coefficient`, `reference_area`, `yaw_inertia`, `roll_inertia`, `pitch_inertia`, `tire stiffness` e `spring stiffness` sono marcati **TBD**.

`value = None` non significa che il database è incompleto in modo sbagliato: significa che il parametro esiste ma il suo valore non è ancora stato determinato. Questo sarà importante quando si sostituirà **TBD** / `Estimated` / `Benchmark` con `Calculated` / `Measured` / `Validated`.

15. ParameterRecord

Struttura dati per rappresentare un parametro ingegneristico:

```
ParameterRecord(
    name,
```

```
    value,  
    unit,  
    source,  
    confidence,  
    status,  
    vehicle_id,  
    configuration_version  
)
```

Esempio:

```
ParameterRecord(  
    name="mass.mass",  
    value=530.0,  
    unit="kg",  
    source="F4 benchmark",  
    confidence="Medium",  
    status="Benchmark",  
    vehicle_id="FDT01",  
    configuration_version="1.0"  
)
```

Scelta importante: non si salva semplicemente `mass = 530`, ma `530 kg` con informazioni su origine, affidabilità, configurazione e stato del dato.

16. Database SQLite

Database gestito tramite `src/fdt/database/`, con struttura:

```
database/  
├─ connection.py  
├─ models.py  
├─ repository.py  
└─ importer.py
```

16.1 connection.py

Gestisce la connessione al database. Definisce `DATABASE_PATH` e `get_connection()`, così i componenti del framework non devono creare manualmente connessioni SQLite ogni volta.

16.2 Tabella parameters

```
id  
name  
value  
unit  
source  
confidence  
status  
vehicle_id  
configuration_version
```

Vincolo fondamentale:

```
UNIQUE(vehicle_id, configuration_version, name)
```

Il database distingue quindi `FDT01 / 1.0 / mass.mass` da `FDT01 / 1.1 / mass.mass`.

16.3 Tabella configurations

```
id  
vehicle_id  
version  
name
```

```
description
parent_version
```

Vincolo: `UNIQUE(vehicle_id, version)`.

Questo permette una vera evoluzione del veicolo:

```
FDT01 V1.0
|
├── V1.1
|
├── V1.2
|
└── V2.0
```

17. Perché configuration_version

Uno dei concetti più importanti dell'architettura: il database non deve dire semplicemente `FDT01 mass = 530 kg`, perché in futuro la massa potrebbe cambiare (es. `V1.0 → 530 kg`, `V1.1 → 525 kg`, `V1.2 → 520 kg`). Bisogna sapere a quale versione appartiene il dato.

```
Vehicle
  ↓
Configuration
  ↓
Parameter
```

18. Repository layer

Concentrato in `src/fdt/database/repository.py`, con funzioni come `initialize_database()`, `insert_parameter()`, `get_parameters()` e gestione delle configurazioni.

18.1 insert_parameter()

Meccanismo di **upsert**: se il parametro non esiste → INSERT; se esiste già → UPDATE. Chiave: `vehicle_id`, `configuration_version`, `name`. Evita duplicazioni.

18.2 get_parameters()

Recupera i parametri di una specifica configurazione, es. `get_parameters("FDT01", "1.0")` restituisce i parametri della FDT01 V1.0.

19. Importer

`database/importer.py` prende la definizione Python della vettura e la porta nel database:

```
FDT01_BASELINE_V1
  ↓
import_vehicle_parameters()
  ↓
ParameterRecord
  ↓
SQLite
```

Importante: la definizione della vettura e la persistenza dei dati sono separate.

20. Script di inizializzazione

Script sotto `scripts/` per inizializzare la baseline:

```
python .\scripts\initialize_fdt01.py
```

Ottiene un database inizializzato con FDT01, V1.0, Baseline e relativi parametri.

21. Problema incontrato nel database

Errore riscontrato:

```
sqlite3.OperationalError: no such column: vehicle_id
```

Causa: il database SQLite esistente era stato creato con una versione precedente dello schema. Dopo la correzione: **9 passed**, poi **10 passed** — parte database tornata completamente operativa.

22. Pint — sistema delle unità

Introdotta **Pint** per non gestire le unità manualmente. Creato `src/fdt/units/` con un registry centrale `ureg`.

Esempi:

```
530 * ureg.kg
130 * ureg.kW
```

con conversioni fisicamente coerenti.

22.1 Problema di fdt.units

Al primo test delle unità: `ModuleNotFoundError: No module named 'fdt.units'`, perché la struttura `fdt/units/` non era ancora stata creata correttamente. Dopo la correzione: **11 passed**.

23. Validation framework

Creato `src/fdt/validation/`:

```
validation/
├── __init__.py
├── parameters.py
├── parameter_dimensions.py
```

23.1 Prima validazione: valori negativi

`validate_parameter()` controlla innanzitutto se il valore è negativo. Esempio: `530 kg` → valido; `-530 kg` → `ValueError`.

23.2 Gestione dei TBD

`value is None` non genera automaticamente un errore, perché `None` può rappresentare **TBD** (es. `drag coefficient = None` è un parametro non ancora determinato, non un errore del framework).

23.3 Secondo livello: dimensionalità

Creato `parameter_dimensions.py` con una mappa, ad esempio:

Parametro	Dimensione
mass.mass	mass
geometry.wheelbase	length
powertrain.maximum_power	power

powertrain.maximum_torque	torque
powertrain.maximum_engine_speed	angular_speed

Questo permette al framework di sapere che tipo di grandezza fisica dovrebbe rappresentare ogni parametro.

23.4 Perché era necessario

Prima della validazione dimensionale, `mass.mass = 530, unit = "m"` poteva teoricamente passare — ma `530 m` è una lunghezza, mentre `mass.mass` deve essere una massa. Obiettivo:

```
mass.mass = 530 kg    ✓
mass.mass = 530 g     ✓
mass.mass = 530 m     ✗
```

23.5 Il test che inizialmente falliva

`test_invalid_mass_unit_is_rejected` doveva verificare `530 m` come massa, ma inizialmente dava:

```
Failed: DID NOT RAISE ValueError
```

Verifiche con Pint hanno confermato che `530 meter → [length]` e `1 kg → [mass]`, quindi Pint riconosceva correttamente le dimensioni.

23.6 Correzione della validazione

Il controllo confronta ora direttamente `quantity.dimensionality` con `expected_unit.dimensionality`:

```
530 m      → [length]
mass.mass   → [mass]
[length] != [mass] → ValueError

130 kW      → [power]
maximum_power → [power]
[power] == [power] → PASS
```

23.7 Non richiediamo un'unità esatta

Non si impone `maximum_power → W` obbligatoriamente, ma `maximum_power → dimensione POWER`. Quindi `130 kW` è valido anche se il database normalmente usa `130000 W`, perché `kW` e `W` sono entrambe unità di `[power]`. Lo stesso principio vale per altre grandezze.

24. Stato attuale dei test

Arrivati a **16 passed**:

Componente	Stato
Database	✓
Vehicle definition	✓
Configurations	✓
Parameter import	✓
Parameter retrieval	✓
Units	✓
Validation	✓
Dimensional checks	✓
TBD handling	✓
Automated tests	✓

La base software è quindi stabile.

25. Cosa abbiamo costruito realmente

Il risultato non è ancora un Digital Twin completo: è stato costruito il **foundation layer**.



Questo è il nucleo sul quale costruire il resto.

26. Cosa NON abbiamo ancora fatto

Non ancora implementato:

- **Vehicle dynamics:** longitudinal, lateral, vertical dynamics, load transfer, braking, acceleration, cornering
- **Powertrain:** engine map, torque curve, power curve, gear ratios, final drive, gearbox, drivetrain losses
- **Tires:** modello longitudinale, laterale, combined slip, load sensitivity, temperatura, stiffness, friction
- **Aerodynamics:** drag, downforce, aero balance, ride-height dependency, aero maps
- **Suspension:** kinematics, motion ratio, spring rates, damper, roll stiffness, camber gain, toe, compliance
- **FEM:** la cartella esiste ma non è ancora integrata nel framework
- **CFD:** stesso discorso
- **Lap time simulation:** non ancora implementata
- **Optimization:** non ancora iniziata la vera ottimizzazione del veicolo

27. Il prossimo livello identificato: Engineering Requirements

Attualmente si sa che `mass.mass → dimension = mass`, ma non sono ancora definiti `minimum` e `maximum`. Esempio di sviluppo futuro:

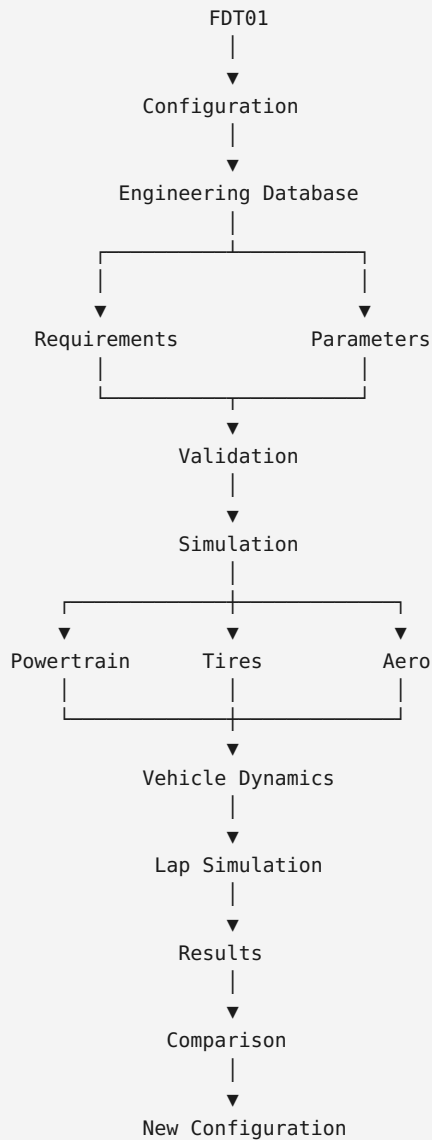
```
mass.mass
  dimension: mass
  unit: kg
  minimum: 300 kg
  maximum: 800 kg

geometry.wheelbase
  dimension: length
  minimum: 1.5 m
  maximum: 4.0 m
```

Questo introdurrà un livello di validazione più potente:

Parameter
↓
Unit validation
↓
Dimension validation
↓
Range validation
↓
Engineering requirement

28. Architettura futura



29. Il concetto fondamentale del progetto

La cosa più importante impostata finora: **non modificare semplicemente i numeri della vettura**. Quando la macchina verrà migliorata, si creeranno nuove configurazioni, ad esempio:

FDT01 V1.0 → Baseline
FDT01 V1.1 → Improved tire model
FDT01 V1.2 → Aerodynamic update
FDT01 V1.3 → Suspension optimization

In questo modo sarà possibile confrontare V1.0 vs V1.1 vs V1.2 vs V1.3 e sapere sempre perché una vettura è diventata migliore o peggiore.

30. Stato attuale in una frase

FormulaCarDigitalTwin is a Python-based engineering framework for the progressive modelling, simulation, validation and optimization of a Formula 4-based vehicle, currently focused on establishing a configuration-driven engineering database, unit-aware parameter management and automated physical validation.

31. Dove siamo nella roadmap

Fase	Avanzamento
1. Project architecture	100%
2. Git / Python setup	100%
3. Vehicle baseline	100%
4. Database foundation	100%
5. Configuration management	100%
6. Unit framework	100%
7. Basic validation	100%
8. Engineering requirements	20%
9. Vehicle dynamics	0%
10. Powertrain model	0%
11. Tire model	0%
12. Aero model	0%
13. Lap-time simulation	0%
14. Optimization	0%

Non significa essere al 20% del progetto complessivo in senso stretto: le parti di simulazione saranno molto più grandi. Ma è stata completata una parte fondamentale — l'infrastruttura sulla quale costruire tutto il resto.

32. In sintesi

Partiti da un repository GitHub vuoto, si è arrivati a:

```
Formula_digitaltwin
|
├─ Git development workflow
├─ Python environment
├─ Structured package
├─ FDT01 vehicle
├─ FDT01 V1.0 Baseline
├─ SQLite database
├─ Parameter records
├─ Configuration records
├─ Parameter importer
├─ Pint unit system
├─ Physical validation
├─ TBD handling
├─ Automated tests
└─ 16 PASSED
```

Questo è un buon punto per fermarsi e documentare: da qui si può iniziare la parte più propriamente ingegneristica — trasformare il database da semplice contenitore di parametri a requirements + engineering knowledge base, e poi collegarlo ai primi modelli fisici della FDT01.